DeepTrace: Learning to Optimize Contact Tracing in Epidemic Networks With Graph Neural Networks

Chee Wei Tan, *Senior Member, IEEE*, Pei-Duo Yu, Siya Chen, and H. Vincent Poor, *Life Fellow, IEEE*

***Abstract*—Digital contact tracing aims to curb epidemics by identifying and mitigating public health emergencies through technology. Backward contact tracing, which tracks the sources of infection, proved crucial in places like Japan for identifying COVID-19 infections from superspreading events. This paper presents a novel perspective on digital contact tracing by modeling it as an online graph exploration problem, framing forward and backward tracing strategies as maximum-likelihood estimation tasks that leverage iterative sampling of epidemic network data. The challenge lies in the combinatorial complexity and rapid spread of infections. We introduce *DeepTrace*, an algorithm based on a Graph Neural Network that iteratively updates its estimations as new contact tracing data is collected, learning to optimize the maximum likelihood estimation by utilizing topological features to accelerate learning and improve convergence. The contact tracing process combines either BFS or DFS to expand the network and trace the infection source, ensuring efficient real-time exploration. Additionally, the GNN model is fine-tuned through a two-phase approach: pre-training with synthetic networks to approximate likelihood probabilities and fine-tuning with high-quality data to refine the model. Using COVID-19 variant data, we illustrate that *DeepTrace* surpasses current methods in identifying superspreaders, providing a robust basis for a scalable digital contact tracing strategy.**

***Index Terms*—Digital contact tracing, contagion source detection, maximum likelihood estimation, graph neural network, online graph exploration.**

I. Introduction

During the early days of the COVID-19 pandemic, a multitude of mobile applications [1], [2], [3], [4] were deployed to detect individuals potentially exposed to the SARS-CoV-2 coronavirus and mitigate its transmission. Digital contact tracing, driven by network analytics and machine learning, continues to play a critical role in epidemic surveillance and preventing hospital-acquired infections by optimizing coverage and tracking the spread of infectious diseases [5]. Its primary objective is to curb the transmission of epidemics, but this is complicated by challenges such as rapid disease spread, asymptomatic carriers, and the disproportionate impact of superspreaders (e.g., approximately 20% of infected individuals account for 80% of transmissions [6]). The development of robust, automated digital contact tracing systems leveraging network science and machine learning remains in its early stages.

It has been recently shown in [2], [7], [8], [9] that compared with 'forward' tracing (i.e., to whom disease spreads), 'backward' tracing (from whom disease spreads) can be more effective due to overlooked biases arising from the heterogeneity in contacts. Tracing sources of spreading (i.e., backward contact tracing), as had been used in Japan [8], has proven effective as going backward can pick up infections that might otherwise be missed at superspreading events. The ability to identify the source of spreading (i.e., the Patient Zero or superspreaders in a pandemic) is essential to contact tracing. In this paper, we propose a novel formulation of the contact tracing problem that integrates both forward and backward tracing, utilizing maximum likelihood estimation as a subproblem. Our approach focuses on *optimizing who to track and how to track* through graph neural network learning, employing network centrality metrics to design fast and efficient contact tracing algorithms.

A. Related Works

The contagion source inference problem was first studied in the seminal work [10] as a maximum likelihood estimation problem based on a Susceptible-Infectious (SI) model [11], which is a special case of the classical Susceptible-Infected-Recover (SIR) model in epidemiology [12], [13], [14]. This problem becomes challenging with large graphs, as the scale complicates tracking spreading likelihoods and calculating permissible permutations for each node. However, the degree-regular tree special case can be solved in polynomial time using a network centrality approach known as rumor centrality [10], [15], which is proportional to the number of permitted permutations. Equivalently, this optimal likelihood estimate is the tree centroid [16], [17], [18]. The network centrality approach [10] has been extended to scenarios like random trees [19], [20], multiple snapshot

Received 13 October 2023; revised 21 June 2024 and 8 November 2024; accepted 28 December 2024. Date of current version 6 February 2025. This work was supported in part by the U.S. National Science Foundation under RAPID Grant IIS-2026982, in part by the National Science and Technology Council of Taiwan under Grant 112-2115-M-033-004-MY2, in part by Hong Kong ITF under Project ITS/188/20, in part by the Singapore Ministry of Education Academic Fund under Grant RG91/22, in part by the National Medical Research Council CS-IRG under Grant CIRG24jul-0021, in part by Sayling Wen Cultural & Educational Foundation Faculty Scholarship, in part by the Institute for Pure and Applied Mathematics Fellowship, and in part by C3.AI Digital Transformation Institute and Princeton Language and Intelligence. The associate editor coordinating the review of this article and approving it for publication was Prof. X. Dong. *(Corresponding author: Pei-Duo Yu.)*

Chee Wei Tan is with the Nanyang Technological University,, Singapore 639798 (e-mail: cheewei.tan@ntu.edu.sg).

Pei-Duo Yu is with the Department of Applied Mathematics, Chung Yuan Christian University, Taoyuan City 32023, Taiwan (e-mail: peiduoyu@cycu.edu.tw).

Siya Chen is with the Department of Computer Science, City University of Hong Kong,, Hong Kong (e-mail: siyachen4-c@my.cityu.edu.hk).

H. Vincent Poor is with the Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: poor@princeton.edu).

Digital Object Identifier 10.1109/TSIPN.2025.3530346

2373-776X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

observations [21], multiple sources [22] and the *epidemic centrality* for a disease pandemic in [23]. Unlike the snapshot model [10], other studies [24], [25], [26], [27] use time series data, Bayesian analysis, and stochastic processes to solve the contagion source inference problem via sequential and quickest detection techniques. For a comprehensive overview of the work on detecting contagion sources, we refer the reader to [18].

However, computing the ML estimator for the source on large epidemic networks can be computationally challenging [10], [23], particularly as the network size grows and complexity increases with the presence of cycles. Thus, using Graph Neural Networks (GNNs) is a viable option. Various studies have used GNNs for pandemic control [28], [29], [30], [31]. Gatta [28] focuses on estimating epidemiological model parameters, differing from other works [29], [30], [31] that aim to identify patient zero (superspreaders). Shah [29] examines SIR and SEIR models with graph convolutional networks, but these are limited to transductive settings and unable to handle graph inputs of varying sizes. In [30], susceptible nodes were pre-labeled as unlikely sources. Meanwhile, [29], [30] used the *true* outbreak source as the label, which can diverge from the maximum-likelihood (ML) estimator. Using the ML estimator as the label, we align training with identifying the node most likely to explain the data, thus reducing bias and improving accuracy.

To address the issues related to the training set [29], [30], and the computational complexity challenges discussed in [15], [23], we propose a GNN-based framework with a two-phase semi-supervised training approach. In this framework, we use graph data with the ML estimator as the superspreader during the fine-tuning phase. Specifically, we adopt the SI spreading model, as thoroughly studied in [15], [18], [23]. It has been established that the ML estimator is equivalent to certain network centers [10], [17], [23]. This equivalence allows us to easily obtain high-quality labeled data for specific network topologies, thereby improving the accuracy of identifying the most likely superspreader.

Lastly, our previous work [31] serves as a pilot study on using GNNs for backward contact tracing, framing contact tracing as online graph exploration. Compared to the conference version, we have studied the performance of the DFS strategy (cf. Lemma 1 and Theorem 2). Additionally, Theorem 3 demonstrates the feasibility of using GNNs to *learn to optimize* maximum likelihood estimators in graphs. In our experimental simulations, we have included numerous tests, evaluating the DFS and BFS strategies combined with various source detection methods from the literature across different network topologies.

B. Main Contributions

- We frame digital contact tracing as an online graph exploration problem, deriving both forward and backward tracing strategies from a maximum likelihood estimation that captures the pandemic's dynamic spread. To address the challenging nonconvex optimization involved, we employ modified graph neural networks to predict network centrality measures. Our low-complexity algorithm, DeepTrace, refines the prediction of the most likely contagion source as data is collected, effectively applying it to epidemic networks of various sizes.
- We utilize GNNs to perform the task of backward contact tracing. By training the GNN to learn to optimize the maximum likelihood estimation (MLE) problem, we enhance its ability to solve the complex nonconvex optimization task of identifying the contagion source. We integrate network centrality methods from the literature for calculating MLE for the source to generate high-quality training data, thereby improving the efficiency and accuracy of our GNN training.
- For the forward contact tracing component, we employed graph theory and algorithmic analysis to thoroughly examine the differences between BFS and DFS strategies. Our experimental simulations demonstrated that using BFS to expand the epidemic network enables faster detection of superspreaders compared to the DFS strategy.
- Using datasets from synthetic epidemic networks and COVID-19 contact tracing in Taiwan and Hong Kong, we show that Algorithm DeepTrace outperforms state-of-the-art heuristics from [10], [23].

II. Contact Tracing Problem Formulation

Let us first introduce the spreading models, including one relevant to COVID-19 in [14], and then present the digital contact tracing optimization problem formulation followed by its mathematical analysis.

A. Epidemic Spreading Model

There are numerous models in the literature in the study of the spread of infectious diseases ranging from commonly well-known ones like the Susceptible-Infectious model to the Susceptible-Infectious-Recovery-Susceptible (SIRS) models [12], [13], [14], [24]. Despite being the most basic one, the SI model can accurately model a pandemic in its early stage (e.g., COVID-19 ca. 2020), whereas other SIRS models [14] incorporate complex factors like virus mutation, societal response (e.g., mask-wearing and social distancing) and the use of vaccines (e.g., COVID-19 ca. 2021).

We primarily focus on the SI model for our analysis, as the characteristics of the COVID-19 virus can persist in individuals for extended periods without a clear recovery or removal phase. Studies have shown that SARS-CoV-2 can remain detectable in patients for several weeks, even after symptoms subside [32]. This persistent detectability supports using the SI model in the early stage of the outbreak. The SI model provides a simplified yet relevant framework for our analytical purposes, focusing on the essential aspects of identifying the initial source of infection. The SI model assumes that there are only two types of individuals, susceptible and infectious, during the epidemic spreading. Also, we follow the assumption that the times between infection events are independent and exponentially distributed [15] with an average infection time (with most estimates of COVID-19 incubation period ranging from 1 to 14 days [14]). The virus spreads in a social contact network can be modeled by a diffusion process on a graph denoted by $\mathbb{G}$. Let $V(\mathbb{G})$ and $E(\mathbb{G})$ denote the set of nodes and edges in $\mathbb{G}$, which represent people and

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

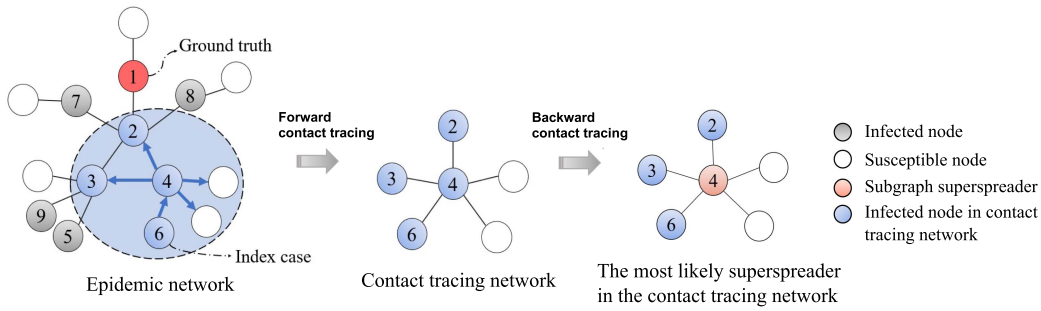


Fig. 1. Illustration of an epidemic network $\mathbb{G}_9$ with nine infections (shaded nodes) whose numbering indicates the infection order starts from the ground truth, i.e., the real superspreader. The contact tracing network G_4 (within a dotted circle) starts from the index case node v_6 (blue arrows show the tracing directions) by forward contact tracing. The backward contact tracing is to find the node in $\mathbb{G}_9$ that is most likely to be the superspreader.

their social contacts (an edge exists between two persons if, say, they are within 6 feet for more than 15 minutes) respectively. We shall call $\mathbb{G}$ the *underlying network*, where the topology of $\mathbb{G}$ poses a constraint to the epidemic spreading [15], [23], [33]. The spreading of the virus starts from a node in $\mathbb{G}$ (i.e., the superspreader) and spreads to other nodes under the constraint that a susceptible node can be infected only if at least one of its neighboring nodes has been infected. At some point in time, all infected nodes in $\mathbb{G}$ form a connected subgraph of $\mathbb{G}$, denoted as $\mathbb{G}_N$ and is called the *epidemic network*, where N represents the number of nodes in the epidemic network.

This epidemic network $\mathbb{G}_N$ and the infection spreading dynamics are assumed to be unknown to a contact tracer, and this is the chief uncertainty faced by all contact tracers, who therefore adopt a strategy to collect this data starting from an index case (i.e., the first documented infected person) and trace his or her close contacts (according to the topology of $\mathbb{G}$) and so on.

B. Forward and Backward Contact Tracing

At various stages of contact tracing, we model an instantaneous snapshot of a subgraph of $\mathbb{G}_N$ that we call the *contact tracing network*. This contact tracing network grows by one (infected) node at each stage. If the traced node is infected, we continue to trace the node's neighbors; otherwise, we stop tracing along a path involving this node. Let G_n denote the contact tracing network with n nodes at the n-th stage of tracing (thus, the index case is G_1). Hence we have $G_n \subseteq \mathbb{G}_N \subseteq \mathbb{G}$. A goal of contact tracing is to find the node in $\mathbb{G}_N$ that is most likely to be the superspreader, as shown in Fig. 1. However, this most likely superspreader may not yet be in G_n, meaning that the contact tracing effort is still in its early stage or not fast enough (relative to the pandemic spreading speed). In such a case, backward contact tracing should yield an estimate as close as possible to this most likely superspreader. In other words, given the available data at the n-th stage, the contact tracer finds the node in G_n that is the fewest number of hops away from the most likely superspreader in $\mathbb{G}_N$ (i.e., the optimal maximum likelihood estimate had this $\mathbb{G}_N$ been given entirely upfront to the contact tracer [10]).

Given the data G_n harvested by contact tracing at the n-th stage, we have the following maximum-likelihood estimation problem:

$$\hat{v} \in \arg \max_{v \in G_n \subseteq \mathbb{G}_N \subseteq \mathbb{G}} \mathbb{P}(G_n \mid v), \tag{1}$$

where $\mathbb{P}(G_n \mid v)$ is the likelihood function and $\hat{v}$ is the most-likely superspreader of the current observed G_n. The key challenge is that $\mathbb{G}_N$ is unknown to the contact tracer, who has to consider:

Forward contact tracing: How to construct the contact tracing network efficiently starting from a given index case?

Backward contact tracing: How to solve (1) to give the best instantaneous estimate of the superspreader given the data?

Answering both the forward and backward contact tracing jointly constitutes an iterative statistical inference process to track the most likely superspreader in the epidemic network. Specifically, for the forward contact tracing subproblem, it is natural to enlarge the forward contact tracing network with breadth-first search (BFS) or depth-first search (DFS) graph traversal starting from the index case. For the backward contact tracing subproblem, suppose that a given node v in G_n is the superspreader. Then, starting from this particular node, there are many possible ways to infect all the other nodes consistent with the given graph G_n harvested by contact tracing at the n-th stage. Even though $\mathbb{G}_N$ is unknown, it is reasonable for a contact tracer to assume a possible infection order in G_n by a *permitted permutation* given as $\sigma = \{v_1 = v, v_2, \ldots, v_n\}$. Thus, we can reformulate problem (1) as:

$$\hat{v} \in \arg \max_{v \in G_n} \sum_{\sigma \in \Omega(G_n|v)} \mathbb{P}(\sigma|v), \tag{2}$$

where $\Omega(G_n \mid v)$ is the collection of all permitted permutations for G_n rooted at v, and $\mathbb{P}(\sigma|v)$ is the probability of the permitted permutation σ with v as source.

Under the SI spreading model used in [10], [15], [23], the probability of a susceptible node being infected in the next time period is uniformly distributed among all outreaching edges from the infected nodes at the boundary of G_n, and the

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

probability of a permitted permutation is as follows:

$$\mathbb{P}(\sigma|v) = \prod_{i=1}^{n-1} \frac{\Phi_i}{\sum_{j=1}^{i}[d(v_j) - 2\Phi_{j-1}]}, \tag{3}$$

where $d(v_i)$ is the degree of node v_i in $\mathbb{G}$, and $\Phi_i = |e(v_{i+1}) \cap (\bigcup_{j=1}^{i} e(v_j))|$ with $e(v_i)$ being the set of edges connecting to node v_i. For the edge case, we define $\Phi_0 = 0$. The value of Φ_i counts the number of edges linked between node v_{i+1} and its infected neighbors. Hence, the probability of being infected is proportional to the number of infected neighbors. Note that the probability defined in (3) also works if $\mathbb{G}$ is a graph with multiple edges, i.e., the number of edges between a given pair of nodes can be more than one.

If $\mathbb{G}$ is a general tree as a special case, then every newly infected node, say v_{i+1} only connected to one of the infected nodes; otherwise, $\mathbb{G}$ is not a tree. Hence, when $\mathbb{G}$ is a general tree, we have $\Phi_i = 1$, for $i \neq 0$, and the probability of a permitted permutation can be given by [10]:

$$\mathbb{P}(\sigma|v) = \prod_{i=1}^{n-1} \frac{1}{\sum_{j=1}^{i} d(v_j) - 2(i-1)}. \tag{4}$$

C. Regular Trees and General Graphs

In this section, we will discuss the differences in using maximum likelihood estimation to find the source estimator when the underlying network is a regular tree versus a general graph. Additionally, we will explore how the known results for regular trees can be applied to general graphs.

From (3), we can observe that when $\mathbb{G}$ is a regular tree, both Φ_i and $d(v_i)$ are constants which implies the value of (3) is also a constant. Thus, the computation of the likelihood only depends on the value of $|\Omega(G_n, v)|$ in (2), which simplifies the computation. On the other hand, it is still unknown whether there exists an efficient method for calculating the likelihood when $\mathbb{G}$ is a general graph since $\mathbb{P}(\sigma \mid v)$ is different for each possible $\sigma \in \Omega(G_n, v)$ and the size of $\Omega(G_n, v)$ grows exponentially with n. To overcome this computational complexity issue, we aim to leverage GNN to find the MLE. Various theoretical results on regular trees or unicyclic regular graphs [10], [23] provide us a way to generate the high-quality dataset; the label of each piece of data is the exact MLE of the epidemic graph for training GNN. The analysis in Theorems 1, 2 shows the best and the worst possible outcomes during the contact tracing on regular trees, which provides us bounds on the performance of the contact tracing on general graphs.

Coming back to the contact tracer starts from the index case G_1 and collects more data in a forward manner (i.e., enlarging G_n in (2)), the contact tracer also predicts the superspreader for that instant by solving (1). Intuitively, this means that as the contact tracing subgraph G_n grows, the contact tracer desires this prediction to be closer (in terms of the number of hops in $\mathbb{G}_N$) to the most likely superspreader in $\mathbb{G}_N$. Considering this, how should G_n grow in size in *forward contact tracing*? We propose to enlarge G_n using BFS or DFS graph traversal [34], [35], which are detailed in the next section.

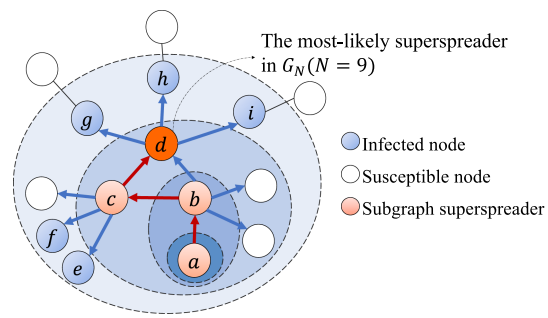


Fig. 2. As the contact tracing network enlarges starting from the index case v_a with BFS traversal, as ordered alphabetically $\{a, b, c, \ldots, i\}$, the most likely superspreader given by (2) moves closer (in terms of number of hops) to the most-likely superspreader v_d in the epidemic network $\mathbb{G}_N$ (indicated by red arrows).

III. Digital Contact Tracing by BFS and DFS

In this section, we detail interesting insights into tracing superspreaders in BFS tracing networks (using BFS traversal for forward contact tracing) and DFS tracing networks (using DFS traversal for forward contact tracing). Mathematically, the forward contact tracing process yields a rooted tree $\mathbb{G}_N$, whose nodes help solve (2). For special cases, Theorem 1 states that this maximizer of (2), i.e., the graph centroid of $\mathbb{G}_N$, converges to the graph centroid of $\mathbb{G}_N$ along a simple path on $\mathbb{G}_N$ when N is fixed.

A. Contact Tracing by Breadth-First Search

Firstly, we propose to enlarge $\mathbb{G}_N$ using the BFS graph traversal algorithm [34], [35]. Fig. 2 illustrates this with a tracer who starts from node v_a and then moves to node v_d (the most likely superspreader) gradually.

Let $\text{dist}(u, v)$ denote the graph distance (i.e., number of hops) between node u and node v. If N is fixed, then we have the following result showing that $\text{dist}(v_n^*, v_N^*)$ is a decreasing sequence as n grows when $\mathbb{G}_N$ is a d-regular tree, where d-regular tree is a tree that all non-leaf nodes have d neighbors.

Theorem 1: If $\mathbb{G}$ is a d-regular tree, and there exists a node $\overline{v}$ such that for any two leaf nodes v_{leaf} and u_{leaf} in $\mathbb{G}_N$,

$$|\text{dist}(v_{\text{leaf}}, \overline{v}) - \text{dist}(u_{\text{leaf}}, \overline{v})| \leqslant 1, \tag{5}$$

then the trajectory of v_n^* is exactly the shortest path from the index case v_1^* to v_N^* in $\mathbb{G}_N$.

Theorem 1 implies that in the special case where $\mathbb{G}$ is a d-regular tree, the epidemic network $\mathbb{G}_N$ satisfies (5). By employing the BFS contact tracing strategy, we can progressively approach the optimal solution v_N^* with each expansion of G_n. Thus, Theorem 1 underscores the effectiveness of the BFS strategy in achieving optimal solutions.

Next, we provide an example, a class of graph structure satisfying (5). If G_n is a complete N-ary tree, then $|\text{dist}(v_{\text{leaf}}, v_n^*) - \text{dist}(u_{\text{leaf}}, v_n^*)| \leqslant 1$ for every v_{leaf} in G_n. Then, from Theorem 1, we have that the trajectory of v_n^* is exactly the shortest path from v_1^* to v_N^* in $\mathbb{G}_N$.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

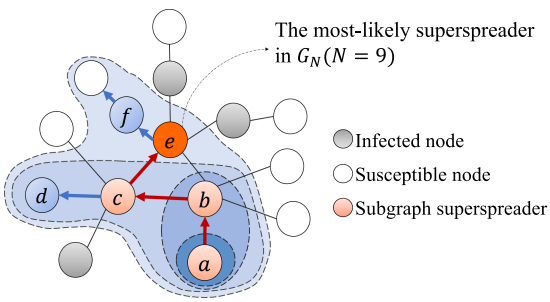


Fig. 3. As the contact tracing network enlarges starting from the index case v_a with DFS traversal, as ordered alphabetically $\{a, b, c, \ldots, f\}$, the most-likely superspreader given by (2) moves closer (in terms of number of hops) to the most-likely superspreader v_e in the epidemic network $\mathbb{G}_N$ (indicated by red arrows).

B. Contact Tracing by Depth-First Search

Let us consider enlarging $\mathbb{G}_N$ with DFS tracing strategy [35], illustrated in Fig. 3. We can categorize all consecutive pairs (v_i^*, v_{i+1}^*) in S into the following three types:

$$S_1 = \{(v_i^*, v_{i+1}^*) | v_i^* = v_{i+1}^*\},$$

$$S_2 = \{(v_i^*, v_{i+1}^*) | v_i^* \neq v_{i+1}^*, \text{ and } v_j^* \neq v_{i+1}^*, \ \forall j < i\},$$

$$S_3 = \{(v_i^*, v_{i+1}^*) | v_i^* \neq v_{i+1}^* \text{ and } v_j^* = v_{i+1}^* \text{ for some } j < i\}.$$

The first type of consecutive pairs in S_1 represent the estimated superspreader that remains the same when the contact tracing network grows from G_i to G_{i+1}. The second type, S_2 implies that the node v_{i+1}^* is computed as the estimated superspreader for the first time. The third type, S_3 implies that v_{i+1}^* had been chosen as the estimated superspreader before. Since the contact tracing network is a subgraph of a degree-regular tree, there are two maximum likelihood estimators when $\mathbb{G}_N$ can be divided into two subgraphs with the same size [15]. We assume for each consecutive pair (v_i^*, v_{i+1}^*) in S_2 and S_3, we have $P(G_{i+1} \mid v_{i+1}^*) > P(G_{i+1} \mid v_i^*)$, i.e., we only select v_{i+1}^* as the new estimator when $P(G_{i+1} \mid v_{i+1}^*) > P(G_{i+1} \mid v_i^*)$. Let $|S_1|$, $|S_2|$, and $|S_3|$ denote the size of these three types of consecutive pairs, respectively. Let T^v be a rooted tree of $\mathbb{G}_N$ rooted at v and denote the subtree of T^v rooted at u as T_u^v. Note that the size of S is N, so there are only $N-1$ consecutive pairs in S. Moreover, S_1, S_2 and S_3 are mutually exclusive. Hence, we can conclude that

$$|S_1| + |S_2| + |S_3| = N - 1. \tag{6}$$

We introduce the following insights of the sequence S by considering S_1, S_2, and S_3.

Lemma 1: Let $\mathbb{G}$ be a d-regular tree, and we apply the DFS tracing strategy, then we have $|S_1| \geqslant |S_2| \geqslant |S_3|$. Moreover, we can bound $|S_1|$, $|S_2|$ and $|S_3|$ as follows:

- $|S_1| \geqslant (N-1)/2 \geqslant |S_2|$,
- $|S_3| \leqslant (N-1)/4$.

The edge cases of Lemma 1 happen when $\mathbb{G}_N$ is a line graph with an odd number of nodes, and the index case is either the center or the leaf node of $\mathbb{G}_N$. We can deduce that more than half of those consecutive pairs in S are type one, i.e., the estimated superspreader will remain the same most of the time. Lastly, this lemma serves as a tool when proving the following theorem.

Theorem 2: Let $\mathbb{G}$ be a d-regular tree and we apply DFS tracing strategy, if $v \in \mathbb{G}_N$ and $(\cdot, v) \in S_3$, then we have $|(\cdot, v)| \leqslant \log_2(N)$, where $|(\cdot, v)|$ denote the number of the pair $(\cdot, v)$ in S_3.

Theorem 2 implies that when using the DFS tracing strategy to build the contact tracing network, each node in the epidemic network $\mathbb{G}_N$ should not be identified as the superspreader more than $\log_2(N)$ times as the contact tracing network expands iteratively. Next, let us compare the performance of both the DFS and BFS contact tracing strategies using two quantifiable metrics (that can be readily measured once the ground truth is given). The first metric is the average error, which is defined by

$$\frac{1}{N} \sum_{i=1}^{N} \text{dist}(v_i^*, v_N^*). \tag{7}$$

The second metric is the first detection time, which is the time when the estimated most-likely superspreader first matches the ground truth and is defined as

$$\min\{i \mid v_i^* = v_N^*\}. \tag{8}$$

The average error metric assesses the performance of any digital contact tracing algorithm, accounting for the selected forward contact tracing strategy and potential suboptimality in maximizing likelihood estimates. On the other hand, the first detection time measures how quickly the contact tracing algorithm can zoom in on identifying the most likely superspreader without a global view of the overall epidemic networks. It is desired that the most likely superspreader is the actual ground truth even as the epidemic network size N increases independently of the digital contact tracing effort.

It is not surprising that the performance of any forward contact tracing strategy depends on the distance between the index case and the most likely superspreader, i.e., $\text{dist}(v_1^*, v_N^*)$. For example, let the epidemic network $\mathbb{G}_{10}$ be a 3-regular tree with six leaves. Assume that the index case is a leaf node, then the pair (average error, first detection time) is (0.7,6) and (0.8,7) for the DFS and BFS strategies, respectively. However, if the index case is exactly the ground truth, then the pair (average error, first detection time) should be (0.4,0) and (0,0) for the DFS and BFS strategies, respectively. If N is fixed and $\mathbb{G}_N$ has a sufficiently small graph diameter, the convergence of the BFS strategy may be slower than that of the DFS strategy because the initial sets of nodes may have more neighbors. On the other hand, when $\mathbb{G}_N$ has a sufficiently large graph diameter, the DFS strategy may converge slower than the BFS strategy. Other graph-theoretic features of $\mathbb{G}_N$, such as degree and cycles, can also affect the choice of the forward contact tracing strategy.

C. Interpretation as Online Maze Solving

We may view splitting (1) into forward/backward tracing as a *maze-solving* process on the epidemic network $\mathbb{G}_N$. Each step taken by the maze solver corresponds to the discovery of a leaf node v^n of the rooted tree $\mathbb{G}_N$ at the nth iteration of the forward contact tracing stage and a corresponding v_n^* of the backward

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

contact tracing. We can treat two aforementioned sequences $S = (v_1^*, v_2^*, \ldots, v_N^*)$ and $X = (v^1, v^2, \ldots, v^N)$ as two trajectories on two mazes respectively. The trajectory S is to find the desired exit v_N^*, and the trajectory X traverses the graph. However, these two mazes are explored and solved at the same time, i.e., once the maze solver takes a step from v^i to v^{i+1}, there is a corresponding movement on S from v_i^* to v_{i+1}^*. If we assume that there is only one contact tracer, and each time a new node is discovered in the $(i+1)$th iteration, the contact tracer has to travel from v^i to v^{i+1} with the distance $\text{dist}(v^i, v^{i+1})$. Then, the goal of the contact tracing is to minimize both the total traveling distance $\sum_{i=1}^n \text{dist}(v^i, v^{i+1})$ of the contact tracer and the first detection time of v_N^* (i.e., the optimal maximum-likelihood estimate had $\mathbb{G}_N$ been given in advance) during the discovery of all infected nodes in $\mathbb{G}_N$. The maze solver (i.e., contact tracer) knows neither the complete maze topology induced by $\mathbb{G}_N$ nor the target v_N^*.

Upon visiting a vertex (i.e., the graph centroid of $\mathbb{G}_N$) in this abstract maze for the first time, the maze solver has information about some of its incident edges only. The contact tracer has a map of all the infected nodes visited and edges connecting them but cannot tell where each leads until $\mathbb{G}_N$ is fully traversed. Both forward and backward contact tracing mirror the process of online graph exploration or maze traversal. For a tree with no cost to discover new nodes, Theorem 1 shows that BFS finds the shortest path (i.e., sequence of centroids), while Theorem 2 indicates DFS offers minimal step variance. Moreover, if $\mathbb{G}_N$ is a tree, these centroids coincide with the rumor and distance centers [17].

Interpreting forward/backward tracing as an online maze-solving problem links (1) to classic online graph exploration [36], [37], whose offline counterpart (Traveling Salesman Problem) is NP-hard. Thus, finding the optimal online solution of (1) is NP-hard. The best-known online algorithm (a DFS variant) achieves a constant competitive ratio of 16 on planar graphs [36], [37]. This viewpoint suggests both low-complexity greedy approaches and more advanced algorithms for contact tracing. Additionally, machine learning can *learn to optimize* such problems at scale [38], [39], e.g., via GNN-based reinforcement learning. When solving (1) online with local data, the maze solver constructs a full map of graph centroids and can integrate maze-solving algorithms like A* [35]. Forward tracing horizons and edge weights can be adapted, and alternate centrality measures (e.g., protection [40] or vaccine centrality [17], Bonacich's centrality [41]) can guide novel forward-backward tracing strategies.

IV. DeepTrace: Learn to Optimize Tracing

In general, solving the NP-hard problem in (2) is computationally challenging, especially when the underlying $\mathbb{G}_N$ is large and with N increasing throughout the contact tracing process. To address this challenge, we leverage GNNs to propose a scalable contact tracing algorithm named *DeepTrace* (see Section IV-E), capitalizing on the above forward and backward contact tracing decomposition. DeepTrace aims to emulate the online graph exploration strategy to maximize the number of previously unseen unique states in a limited number of steps by employing a graph-structured memory to encode the past trajectory of the contact tracer and hidden structures in the spreading model and network topologies.

A. Overview of GNN-Based Framework

As a deep learning model for processing unstructured data, a GNN extracts graph features and information from the input data encoded as graphs. The learning mechanism of the GNN is to iteratively aggregate features and information from neighboring nodes for each node in the input graphs. Aggregating information from neighbors is equivalent to a message-passing process among nodes in a graph. Lastly, we update the values of the learning parameters for regression or classification tasks [42], [43]. To effectively capture both the statistical and graph topology attributes inherent in network-structured input data, the training stage of the GNN in semi-supervised learning is crucial. Specifically, the (3) and (4) can act as node label generators during the construction of fine-tuning datasets for the proposed algorithm.

To apply GNNs to solve (2), we need to consider two key computational aspects: the underlying epidemic network $\mathbb{G}_N$ is unknown to the contact tracer, and the size of $\mathbb{G}_N$ can be potentially massive. The role of the GNN is, therefore, to facilitate the computation of a (possibly suboptimal) solution to (2) when $\mathbb{G}_N$ is large. Due to the adaptability of GNNs, we can first generate a training set using small graphs (e.g., tens or hundreds of nodes) that are subgraphs of $\mathbb{G}_N$ and augment the descriptors with the structural features for each node of these graphs as the input data. The training stage requires that correct labels (that is, exact values of (3) be associated with each of these small graphs. As the size of the graph grows, the computational cost of (3) becomes expensive. We can instead approximate (3) by sampling method to efficiently compute labels for training data. Finally, we train the GNN model with the input subgraph data iteratively to update the neural network hyperparameters, as shown in Fig. 4. We explain GNN architecture and its training in the following section.

B. Pre-Training and Fine-Tuning GNNs

The core idea for solving the backward contact tracing problem described in (2) is to predict the value of $\sum_{\sigma\in\Omega(\mathbb{G}_n|v)} \mathbb{P}(\sigma|v)$ for each $v \in \mathbb{G}_n$ using GNNs. This prediction task can be seen as a node label prediction where the label for each node is a continuous value $\mathbb{P}(\mathbb{G}_n \mid v) = \sum_{\sigma\in\Omega(\mathbb{G}_n|v)} \mathbb{P}(\sigma|v)$. In practice, we use the log-likelihood $\log(\mathbb{P}(G_n|v))$ as the node label to avoid arithmetic underflow, as the values of likelihood probabilities can be extremely small as the infected network grows. We employ GraphSage [43], a widely-used inductive GNN model adaptable to varying-sized networks. We use an LSTM-based [43] aggregator to gather information from the neighbors of each node, allowing the model to effectively integrate neighboring node features for learning accurate contact tracing. The lth layer of the GNN model is defined as follows:

$$\mathbf{h}_{N_{\mathbb{G}_N}(v)}^{(l)} = \text{LSTM}(\{\widehat{\mathbf{w}}^{(l-1)}, \mathbf{h}_u^{(l-1)} : u \in N_{\mathbb{G}_N}(v)\}),$$
$$\mathbf{h}_v^{(l)} = \max\left(0, \check{\mathbf{w}}^{(l)} \cdot \left[\mathbf{h}_v^{(l-1)}; \mathbf{h}_{N_{\mathbb{G}_N}(v)}^{(l)}\right]\right), \tag{9}$$

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

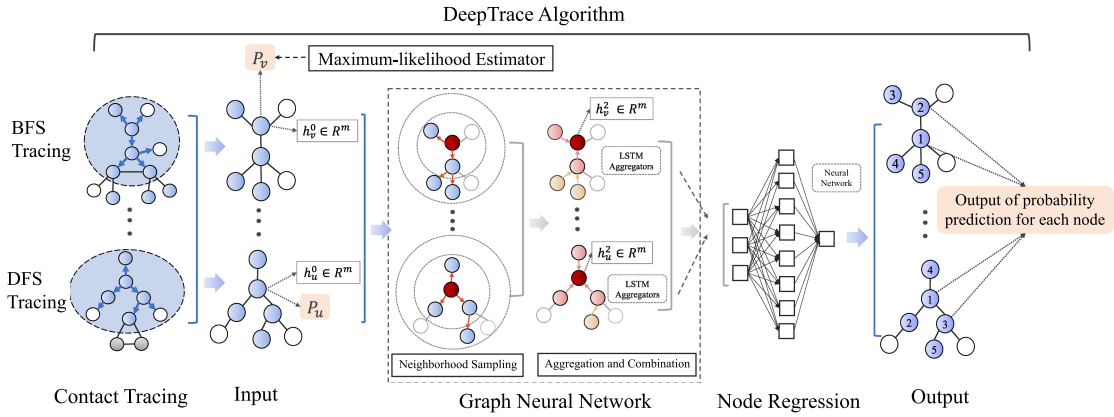


Fig. 4. The overall architecture of Algorithm DeepTrace employs a GNN, taking as input several small-scale networks obtained through BFS and DFS methods. Each node in these networks has structural features and a training label representing the permitted permutation probability. Semi-supervised learning is conducted using GraphSage with LSTM aggregators.

where $\widehat{\mathbf{w}}^{(l)}$ and $\check{\mathbf{w}}^{(l)}$ are the learning parameters in the LSTM aggregators and combination function, respectively. The initial node feature vector is $\mathbf{h}_v^{(0)} = [\mathbf{1}, \hat{r}(v), \check{r}(v)]^T$ for $v \in \mathbb{G}_N$, which we discuss in the Section IV-D. To train the GNN model in DeepTrace during the two-phase training process, we define the loss functions for the pre-training and fine-tuning phases, respectively, as follows:

$$L_p(\widehat{\mathbf{w}}, \check{\mathbf{w}}, v \mid v \in G_n) = \sum_{v \in G_n} \left| \log(\widetilde{\mathbb{P}}(G_n \mid v)) - \mathbf{h}_v^{(L)}(\widehat{\mathbf{w}}, \check{\mathbf{w}}) \right|^2,$$

and

$$L_f(\widehat{\mathbf{w}}, \check{\mathbf{w}}, v \mid v \in G_n) = \sum_{v \in G_n} \left| \log(\mathbb{P}(G_n \mid v)) - \mathbf{h}_v^{(L)}(\widehat{\mathbf{w}}, \check{\mathbf{w}}) \right|^2,$$

where $\widetilde{\mathbb{P}}(G_n \mid v)$ is the approximation of $\mathbb{P}(G_n \mid v)$, $\widehat{\mathbf{w}} = (\widehat{\mathbf{w}}^{(1)}, \ldots, \widehat{\mathbf{w}}^{(L)})$ and $\check{\mathbf{w}} = (\check{\mathbf{w}}^{(1)}, \ldots, \check{\mathbf{w}}^{(L)})$. The effectiveness of backward contact tracing using GNN (9) is proved as follows.

Theorem 3: For a given epidemic network $\mathbb{G}_N = (V, E)$, denote $\text{diam}(\mathbb{G}_N)$ as the diameter of $\mathbb{G}_N$, and define each layer of the GNN model by (9). Then for $\forall\, \epsilon > 0$ there exist a parameter setting $(\widehat{\mathbf{w}}^*, \check{\mathbf{w}}^*)$ in (9) with at most $L = \text{diam}(\mathbb{G}_N) + 1$ layers such that

$$|\mathbf{h}_v^{(L)} - \mathbb{P}(\mathbb{G}_N \mid v)| < \epsilon, \forall\, v \in V.$$

Based on the classical result on the learning capability of feedforward neural networks [44], Theorem 3 states that the value of $\mathbb{P}(\mathbb{G}_N \mid v)$ can be approximated by GNNs with at most $\text{diam}(\mathbb{G}_N) + 1$ layers within an arbitrary error ϵ. From an algorithm design perspective, Theorem 3 establishes the feasibility of approximating $\mathbb{P}(\mathbb{G}_N \mid v)$ using GNNs, providing the theoretical foundation for our subsequent approach to learning $\mathbb{P}(\mathbb{G}_N \mid v)$ with GNNs.

Since obtaining labeled datasets for training (9) is computationally challenging, we employ a two-phase semi-supervised training approach to tune the learning parameters in (9): a pre-training phase followed by a fine-tuning phase. In the pre-training phase, we train the GNN in Deeptrace using a dataset of contact tracing networks with approximations of $\mathbb{P}(G_n|v)$ as labels for each node. This dataset can be easily obtained, as described in Section IV-C. This pre-trained GNN exhibits a great enhancement in performance when compared to a GNN with randomly initialized learning parameters. Then, in the fine-tuning phase, we proceed to adjust the learning parameters in the pre-trained model using an additional dataset of contact tracing networks that contain the exact values of $\mathbb{P}(G_n \mid v)$ as labels for each node in the networks. This dataset can include special cases of regularly-sized networks, such as d-regular graphs [10], [17], as well as a small number of contact tracing networks that have the exact values of $\mathbb{P}(G_n \mid v)$ obtained by calculating probabilities of all the permitted permutations in (2) as labels for each node. Acquiring the latter dataset requires more effort and is considered highly valuable. Therefore, it is not wasted on training the GNN from scratch but rather on fine-tuning the pre-trained model to further enhance its accuracy. This two-phase training process enables us to effectively utilize both the dataset with approximate values of $\mathbb{P}(G_n \mid v)$ and the dataset with exact values of $\mathbb{P}(G_n \mid v)$. Thus, we can attain a GNN that exhibits enhanced performance while reducing the time and computational resources required for training.

C. Data Construction & Annotation for Two-Phases Training

To construct the training datasets for pre-training and fine-tuning the GNN model, we select six different synthetic networks as the underlying network $\mathbb{G}$ with 3500 nodes. The corresponding parameters for each type of synthetic network are detailed as follows: Erdös Rényi (ER) random graphs, characterized by the probability of edge formation between nodes of 10^{-3}; Barabási-Albert (BA) random graphs, where each new node connects to existing nodes by selecting one edge; Watts-Strogatz (WS) random graphs, in which each node connects to one nearest neighbor with a reconnection probability of 0.1; random 3-regular networks; stochastic block model (SBM) networks, defined by three communities, with an intra-community

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

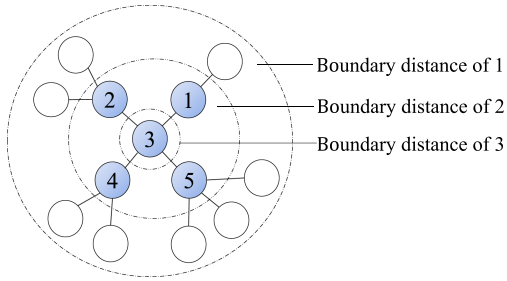


Fig. 5. Illustration of the boundary distance for each node on G_n.

edge formation probability of 2^{-4} and an inter-community edge formation probability of 5×10^{-4}; and sensor networks [45], which cover an area of $55\,\text{km}^2$ with a communication range of 130m.

Next, we apply the SI spreading model [10], [23] on these synthetic networks to simulate the spread of infection to generate the epidemic network $\mathbb{G}_N$. Initially, we uniformly select a source node to initiate the spread of the virus on $\mathbb{G}$. The probability of a node in the graph boundary $\mathcal{B}(G_n)$ being infected in the next time period is uniformly distributed among all outreaching edges from the infected nodes. For example, in Fig. 5, had v_3 been the source, the probability of v_1, v_2, v_4, and v_5 being the second infected node is $1/4$, since there are 4 outreaching edges from v_3. This process continues until the number of infected nodes reaches our predetermined threshold.

1) Dataset Construction and Annotation for Pre-Training: We generate 500 underlying networks $\mathbb{G}$ with 3500 nodes, where each $\mathbb{G}$ is uniformly selected from one of the six types of synthetic networks described above. For each $\mathbb{G}$, we construct $\mathbb{G}_N$ according to the SI model with N randomly chosen from $50 - 1000$. For the labeled training data in the pre-training phase, we can use an approximate estimator of (1) to efficiently obtain $\widetilde{\mathbb{P}}(G_n \mid v)$ for each node v in the epidemic networks, minimizing time and effort. When the epidemic networks are degree-irregular trees, the ML estimator in (1) aims to calculate the sum of all probabilities of permitted permutations. We can rewrite (1) as follows:

$$v \in \arg\max_{v \in G_n} \overline{\mathbb{P}}(G_n \mid v)|\Omega(G_n \mid v)|, \tag{10}$$

where $\overline{\mathbb{P}}(G_n \mid v)$ is the average of the probabilities of all permitted permutations. We can calculate $|\Omega(G_n \mid v)|$ using a message-passing algorithm from [46] with a time complexity of $\mathcal{O}(N)$. However, obtaining the exact value of $\overline{\mathbb{P}}(G_n \mid v)$ is challenging, as it requires identifying all the permitted permutations of $\mathbb{G}_N$ and their corresponding probabilities. Therefore, we randomly select a small sample of the permitted permutations, denoted by $\widetilde{\Omega}(G_n \mid v)$, and approximate $\overline{\mathbb{P}}(G_n \mid v)$ by averaging their probabilities. Thus, the approximate MLE problem is formulated as follows:

$$\begin{aligned}\hat{v} &\in \arg\max_{v \in G_n} \frac{1}{|\widetilde{\Omega}(G_n \mid v)|} \sum_{\sigma \in \widetilde{\Omega}(G_n|v)} \mathbb{P}(\sigma \mid v)|\Omega(G_n \mid v)| \\ &= \arg\max_{v \in G_n} \widetilde{\mathbb{P}}(G_n \mid v). \end{aligned} \tag{11}$$

2) Dataset Construction and Annotation for Fine-Tuning: Since we already have a pre-trained GNN model, we can further refine it using a small amount of high-quality data. Specifically, this involves using data from epidemic networks with exact likelihood probabilities assigned to each node. We generate 250 underlying networks $\mathbb{G}$ with 3500 nodes, where each $\mathbb{G}$ is uniformly selected from one of the six types of synthetic networks described above. For each $\mathbb{G}$, we construct $\mathbb{G}_N$ according to the SI model with a fixed size of $N = 50$. To construct a piece of labeled data with exact likelihood probability for fine-tuning, a natural idea is to calculate the likelihood probability of each node directly by using (3) or (4). We first generate an epidemic network $\mathbb{G}_N$ with N nodes, then for each node $v \in \mathbb{G}_N$ we generate all permitted permutation $\sigma \in \Omega(\mathbb{G}_N \mid v)$ starting from v and compute $\sum_{\sigma \in \Omega(G_n|v)} \mathbb{P}(\sigma|v)$ using (3). Furthermore, we can construct data samples for which exact likelihood probabilities are easily obtainable using (4), such as the d-regular tree. In this case, since the probabilities of all permitted permutations are identical, it is evident from (10) that the likelihood probability for a node in a d-regular tree, as given by (2), is proportional to the epidemic centrality in [17], which can be computed using the message-passing algorithm described in [17].

Lastly, we repeat the data generation procedure in the pertaining phase to generate 250 underlying networks and corresponding epidemic networks for the testing set.

D. Node Features Construction

To ensure the effectiveness of GNN learning for solving (2) in *Algorithm DeepTrace*, it is crucial to design the node features meticulously. This careful design enables the extraction of node information from the underlying epidemic network $\mathbb{G}_N$, corresponding to a specific spreading model during the training phase. We now consider several node features, such as the proportion of instantaneously infected nodes and boundary distances within the graph. Notably, constructing these features requires only linear computational complexity, $O(N)$, as illustrated in Fig. 5.

Infected proportion: The ratio of the number of infected neighbors of a node v_i to the number of all its neighbors:

$$\hat{r}(v_i) = \frac{\hat{d}(v_i)}{d(v_i)},$$

where $\hat{d}(v_i)$ is the number of infected neighbors of the node v_i. For example, the number of infected neighbors of nodes v_1, v_2, v_3, v_4, and v_5 in Fig. 5 are 1,1,4,1 and 1, leading to an infected proportion of nodes v_1, v_2, v_3, v_4, and v_5 being $\frac{1}{2}, \frac{1}{3}, 1, \frac{1}{3}$, and $\frac{1}{4}$, respectively. This feature represents each node's ability to infect new individuals. Intuitively, a node with a lower infected proportion has a higher probability of infecting new individuals and a smaller probability of being the source. Moreover, this particular node feature aims to capture the effect of the particular node position in the permitted permutation in (3). A node with a lower infected proportion ratio tends to be located at the front of the permitted permutation order, resulting in a smaller permutation probability. For example, as in Fig. 5, the different positions of Node 5 for the permitted permutation $\{3, 1, 2, 4, 5\}$

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

and $\{5, 3, 1, 2, 4\}$ result in a probability of 1.7857×10^{-3} and 1.0204×10^{-3}, respectively.

Boundary distance ratio: As the infectivity of the virus evolves–newly symptomatic infectors generally exhibit higher infectivity, whereas long-term symptomatic infectors display lower infectivity–we assume that nodes at the boundary of the epidemic network are newly symptomatic infectors, while those further from the boundary are long-term symptomatic infectors. This characteristic is captured by the boundary distance ratio $\check{r}(\cdot)$. To define $\check{r}(\cdot)$, we first define the graph boundary of G_n as $\mathcal{B}(G_n) = \{u \notin V(G_n) | \exists v \in G_n \text{ s.t. } \text{dist}(u, v) = 1\}$ and the boundary distance $b(v)$ of node v on G_n as $b(v) = \min_{u \in \mathcal{B}(G_n)}\{\text{dist}(u, v)\} + 1$. The boundary distance ratio of v on G_n can now be defined as

$$\check{r}(v) = \frac{b(v)}{\max_{v_j \in G_n} b(v_j)}.$$

For example, let $V(G_n) = \{v_1, v_2, v_3, v_4, v_5\}$ as shown in Fig. 5, and $\mathcal{B}(G_n)$ are those unlabeled nodes. Then, we have $b(v_1) = 2$ and $b(v_3) = 3$. Thus, $\check{r}(v_1) = \frac{2}{3}$, and $\check{r}(v_3) = 1$. Notice that we grow G_i using tree traversal algorithms; hence G_i can be treated as a tree, and the boundary distance ratio of each node in G_i is well-defined. From (2), we see that finding the source also requires information on the size of the collection of all permitted permutations $\Omega(G_n \mid v)$. Notice that a node with a larger boundary distance ratio tends to have more permitted permutations. For example, there are 24 permitted permutations when we choose node v_3 to be the source, but only 6 permitted permutations if we choose either node v_1, v_2, v_4 or v_5 to be the source.

E. The DeepTrace Algorithm

Inspired by the general framework discussed in Section IV-A, we divide the DeepTrace algorithm into two main phases: training and prediction. In the training phase, the GNN model is first pre-trained on contact tracing networks where node labels approximate the likelihood function $\widetilde{\mathbb{P}}(G_n|v)$. This step allows the GNN to gain an initial understanding of networked structures and learn how to approximate the infection source probabilities using these approximated labels. The second step involves fine-tuning the model using the exact values of $\mathbb{P}(G_n|v)$. This fine-tuning process refines the predictive capabilities of our model, enabling it to produce more accurate predictions. As the model is trained on precise source probabilities, it adjusts its parameters to optimize performance in the context of source detection. The inputs for training the GNN during the pre-training and fine-tuning phases consist of the corresponding networks that include node features. For testing the trained GNN, the inputs are the contact tracing networks from the testing dataset. The outputs from all phases–pre-training, fine-tuning, and testing are the predicted log-likelihood of each node being the source. We then find the node with the maximum output value as the predicted source.

In the prediction phase, the algorithm begins by generating the contact tracing network G_n at the n-th stage, using either BFS or DFS. Node features $\mathbf{h}_v^0$ for each node $v \in G_n$ are initialized

Algorithm 1: DeepTrace.

Input: The index case (i.e., G_1); the GNN model defined in (9) with randomly initialized parameters and contact tracing data.

Output: The log-likelihood for every node in the network to be the contagion source.

Training:

1. Pre-train the GNN model using contact tracing networks with node labels as the approximations of $\log \widetilde{\mathbb{P}}(G_n|v)$ in (11).
2. Fine-tune the GNN model using contact tracing networks with node labels having the exact values of $\log \mathbb{P}(G_n|v)$ in (1).

Prediction:

3. **for** $n = 2$ **to** N **do**
   - Generate the contact tracing network G_n using either the BFS or DFS search strategy at the n-th stage;
   - Construct the node features $\mathbf{h}_v^0 = [\mathbf{1}, \hat{r}(v), \check{r}(v)]^T$ for each node in G_n;
   - Predict the source probability for each node in G_n using the fine-tuned GNN model in Step 2;

end

using $\hat{r}(v)$ and $\check{r}(v)$. These node features are then input into the fine-tuned GNN model, which predicts the likelihood of being the source for each node in G_n as shown in Fig. 4. By leveraging the trained GNN model, the algorithm efficiently computes the likelihood of each node being the source of the infection. The prediction phase allows the algorithm to trace the source in the epidemic network.

As contact tracing progresses, Algorithm 1 ultimately identifies the superspreaders for the entire epidemic network. The computational complexity of contact tracing in Algorithm 1 can be decomposed into three main components: sampling complexity, aggregation complexity, and the complexity of the final regression layer. Suppose there are N nodes in the epidemic network. During the sampling procedure, the GNN samples a fixed-size neighborhood for each node. If each node samples S neighbors and this process is repeated for L layers, then the total number of nodes sampled for each node is $O(S^L)$. In the final step of contact tracing, the contact tracing network contains N nodes, so the sampling complexity for this network is $O(NS^L)$. Since we initiate contact tracing from the index case, the number of nodes in the contact tracing network increases sequentially. Consequently, the overall sampling complexity for the entire contact tracing process across the epidemic network is $O(N^2S^L)$. For the aggregation procedure, the GNN aggregates information from its neighbors sampled. Assume that the feature dimension is m. Since the LSTM aggregator processes the features of the sampled neighbors, the complexity of the aggregation step is $O(mS^2)$ per node per layer. Thus, the aggregation complexity for contact tracing the entire epidemic network is $O(N^2LmS^2)$. In the final regression layer, a fully connected layer maps node embeddings to target values, representing the

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DATASETS OF THE SYNTHETIC EPIDEMIC NETWORKS FOR PRE-TRAINING, FINE-TUNING, AND TESTING WHEN TRAINING THE GNN IN DEEPTRACE. THE FIRST COLUMN IS THE NUMBER OF NETWORKS USED IN EACH PHASE

No.	$\lvert V(\mathbb{G}_N)\rvert$	$\lvert E(\mathbb{G}_N)\rvert$	Labels	Task
500	50∼1000	49∼999	$\log(\tilde{\mathbb{P}}(G_n \mid v))$	Pre-training
250	≈ 50	≈ 50	$\log(\mathbb{P}(G_n \mid v))$	Fine-tuning
250	50∼1000	49∼999	$\log(\mathbb{P}(G_n \mid v))$	Testing

likelihood of nodes being superspreaders. The per-node complexity is $O(m)$, resulting in $O(N^2m)$ for the whole network. Thus, the overall computational complexity of contact tracing in Algorithm 1 is $O(N^2LmS^2)$.

V. PERFORMANCE EVALUATION

In this section, we first provide simulation results on the performance of the approximate ML estimator (11). Secondly, we compare the BFS and DFS strategies by simulating the procedure of forward and backward contact tracing. Lastly, we demonstrate the effectiveness of *DeepTrace* on both synthetic networks and real-world networks based on COVID-19 contact tracing data in Hong Kong and Taiwan. As mentioned in Section I-A, the methods proposed in [29], [30] are based on the assumptions of either SEIR or SIR spreading model, which is different from the SI model considered in this paper. We thus focus on baseline comparison relevant to the SI model, namely the rumor center heuristic in [15], and the SCT (Statistical Distance-Based Contact Tracing) algorithm in [23]. The software for *Algorithm DeepTrace* and experimental data are available at https://github.com/convexsoft.

A. DeepTrace for Synthetic Networks

We first evaluate the performance of Algorithm DeepTrace for identifying superspreaders within epidemic networks, using the testing datasets, after we pre-train the GNN model for 150 epochs. Then, we evaluate the performance of DeepTrace again after another 150 epoch training for fine-tuning. There are 250 pieces of data in the testing dataset. The generation of each piece of data follows the procedure described in Section IV-C1 except the node label in the testing dataset is the true log-likelihood $\log \mathbb{P}(G_n|v)$. We summarize the dataset information, including datasets for pre-training, fine-tuning, and testing, in Table I.

We use the bias [31] and a top-k accuracy to evaluate the performance of the trained model in the two-phase training process. The "top-k" metric signifies that the trained GNN identifies the approximate superspreader as one of the k nodes with the highest probability of being superspreaders. This metric is similar to the `pass@k` metric for evaluating the functional correctness of code generated by pre-trained models in [47]. In particular, the `pass@k` metric is the probability that at least one of the top-k generated code samples for a problem passes the unit tests [47]. In the context of evaluating the efficacy of pre-trained models in learning to optimize, this metric measures how well the ranked nodes in G_n solve (1) based on the notion of network centrality. For example, if the approximate superspreader detected by the trained GNN is in the top-10 set, it indicates that this node is among the 10 most likely superspreaders. Thus, the lower the value of k in the top-k set, the higher the likelihood that the detected node is the most probable superspreader. Moreover, a higher likelihood of the optimal solution to (1) being among the top-k for a smaller k implies the better performance of the pre-trained GNN. Thus, the "top-k" metric effectively measures the frequency with which the global optimal solution of (1) appears within the top-ranked prediction of the pre-trained GNN, offering a quantitative assessment of the accuracy.

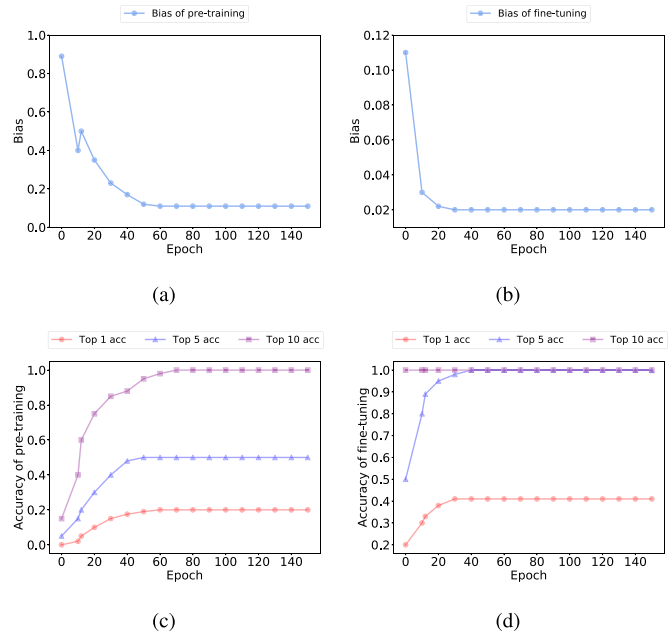


Fig. 6. Illustration of the results of the GNN model: (a) and (b) depict the bias trajectory between the approximate likelihood probability obtained by the GNN model and the accurate likelihood probability for the testing dataset during the pre-training and fine-tuning phases, respectively. (c) and (d) show the trajectories of top-1, top-5, and top-10 accuracies for the testing dataset during the pre-training and fine-tuning phases, respectively.

Fig. 6 illustrates the value of bias and accuracy over the training epochs during both the pre-training and fine-tuning phases. Fig. 6(a) and (a) show that the bias decreases significantly as the number of epochs increases, with a faster and more pronounced reduction during fine-tuning. Initially, the pre-training phase exhibits higher bias, which gradually decreases and levels off after about 20 epochs, whereas fine-tuning achieves a lower bias within the first few epochs. Fig. 6(b) and (b) present the top-1, top-5, and top-10 accuracy trends, demonstrating that accuracy improves steadily throughout both phases. During pre-training, accuracy increases more slowly, with top-10 accuracy achieving the highest values, followed by top-5 and top-1 accuracy. In contrast, fine-tuning rapidly enhances accuracy, reaching a stable level within 20 epochs. These results indicate that fine-tuning not only accelerates convergence but also achieves a more optimal balance between bias reduction and accuracy improvement compared to the model with only the pre-training phase.

Another key evaluation is to measure the generalizability of *DeepTrace Algorithm*, that is, to validate the effectiveness of the trained model by the small-scale epidemic networks in the larger epidemic networks. We construct six additional synthetic

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

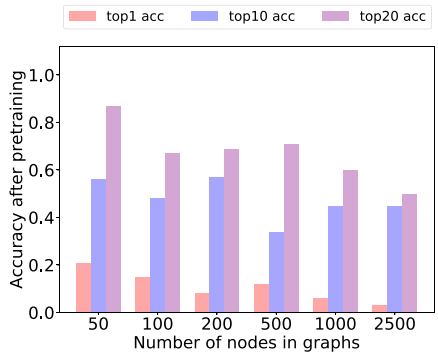


Fig. 7. The top-1, top-10, and top-20 prediction accuracies for validation datasets after pre-training in Algorithm DeepTrace.

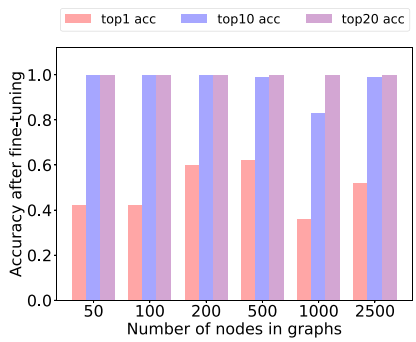


Fig. 8. Validation dataset accuracy metrics at top-1, top-10, and top-20 prediction levels after fine-tuning in Algorithm DeepTrace.

network validation datasets, each with 100 random 3-regular networks, and the number of nodes in each set of networks is $50, 100, 200, 500, 1000$, and 2500, respectively. We use the SI model to simulate the spreading for a given number of infected nodes to generate the infection graphs (see Section IV-C). We evaluate the trained GNN's performance on validation datasets using top-k accuracy.

After training for 150 epochs, we evaluate the prediction accuracy at top-1, top-10, and top-20 for each validation dataset. The results of the pre-training and fine-tuning processes are illustrated in Figs. 7 and 8, respectively. Fig. 7 displays the accuracy trends during the pre-training phase, showing a steady improvement in performance as the model learns. In contrast, Fig. 8 depicts the fine-tuning phase results, highlighting how the model adjusts and refines its predictions with additional training. These figures demonstrate that the training accuracy remains relatively stable, not deteriorating significantly even as the number of nodes in the epidemic networks increases. This demonstrates the GNN's robustness and adaptability across network sizes, highlighting the utility of training on small networks for generalization to larger ones. Training on smaller networks reduces time and enables transfer learning for source detection in larger networks.

B. Evaluation on BFS and DFS Tracing

In this section, we compare the performance of BFS and DFS tracing strategies using first detection time and average error as evaluation metrics, defined by (7) and (8), respectively. Here, we consider five groups of epidemic network datasets with distinct topological structures. The relevant statistics for these data are displayed in Table II. In the final column, we use d to represent the degree of the regular network, p to indicate the likelihood of edge formation between nodes in the ER random network, and $NC/p/q$ to denote the number of communities, the probability of edge formation within a community, and the probability of edge formation between communities, respectively. Additionally, we define the sensor network's coverage area as $Area$ and the communication range, denoted as CR, as the utmost distance between sensors that can form an edge. Let us describe the simulation experiment process. In synthetic networks, we first establish the underlying network $\mathbb{G}$ using different random network models. Next, based on the SI model described in Section II-A, we uniformly and randomly select a source node and begin simulating the spread of the virus on $\mathbb{G}$. Since the number of nodes in our real-world network data is around 100, we stop this spreading process once the number of infected nodes reaches 100, resulting in a 100-node epidemic network $\mathbb{G}_N$. To ensure fairness (as different index nodes may lead to significant performance differences), each node in $\mathbb{G}_N$ is selected as the index node to start the contact tracing process.

We perform forward contact tracing using BFS or DFS and employ Algorithm DeepTrace along with two other methods, Rumor Center [15], and SCT [23], for backward contact tracing. As we expand the contact tracing network G_n, each time a new node is added, we compute the current estimated superspreader in G_n and measure the distance between this estimated superspreader and the true source, noting the error in hops at that moment (cf. (7)). When the estimated superspreader first corresponds to the true source, we record the current size of G_n as the first detection time. When $G_n = \mathbb{G}_N$, we will have identified 100 approximated superspreaders and their corresponding error hops. We calculate the average of these 100 error hops as the average error hops for this simulation experiment. This experiment is repeated 100 times on the same underlying network $\mathbb{G}$, but with different epidemic networks $\mathbb{G}_N$, recording the average error hops, first detection time, and the time spent. We initiate the contact tracing simulation directly on this data for the real-world network data (Taiwan and Hong Kong COVID-19 datasets), as these networks already represent epidemic conditions.

The theoretical result we obtained in Theorem 1 for the BFS strategy describe the distance between the approximated superspreader and the MLE, whereas the theoretical results from Lemma 1 and Theorem 2, for the DFS strategy calculate the upper bound of the number of nodes that will be marked as approximated superspreaders during the contact tracing process. The theoretical results for BFS and DFS are entirely different in scope, so we cannot determine which strategy is better based on theory alone. In our simulation experiments, we tested six combinations of DeepTrace, Rumor Center, and STC with either BFS or DFS and recorded the first detection time and the average distance error in Table III. We can observe that DeepTrace+BFS outperforms other methods from the perspective of the first detection time. Additionally, within Rumor Center and STC, the BFS strategy generally outperforms the DFS strategy. On the other hand, from the perspective of the average distance

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF THE DATASETS USED FOR PERFORMANCE COMPARISON WITH BFS AND DFS TRACING

Epidemic network	Underlying graph	$\lvert\mathbb{G}_N\rvert$	Components	Other parameters
Complete N-ary graph	Nodes = 3500, Edges = 3499	100	1	$N = 3$
Regular tree	Nodes = 3500, Edges = 3499	100	1	$d = 3$
ER random graph	Nodes = 3500, Edges = 6016	100	1	$p = 10^{-3}$
SBM network	Nodes = 3500, Edges = 8086	100	1	$p = 5{\times}10^{-4}$, $q = 2{\times}10^{-4}$, NC=3
Sensor network	Nodes = 3500, Edges = 5710	100	1	Area=$55km^2$, CR=$130m$
Real-world network (Taiwan)	Nodes =92, Edges =117	92	1	–

TABLE III
COMPARISON OF AVERAGE FIRST DETECTION TIME AND AVERAGE DISTANCE ERROR FOR DEEPTRACE AND BASELINE METHODS ACROSS DIFFERENT EPIDEMIC NETWORKS, USING BFS AND DFS TRACING STRATEGIES

Epidemic Network	Average First Detection Time						Average Distance Error					
	DeepTrace		Rumor Center		SCT		DeepTrace		Rumor Center		SCT	
	DFS	BFS	DFS	BFS	DFS	BFS	DFS	BFS	DFS	BFS	DFS	BFS
Complete N-ary graph	34.00	**7.17**	34.81	8.18	38.60	16.95	**0.84**	1.54	1.35	2.31	1.04	1.66
Regular tree	36.10	**16.14**	63.06	18.21	64.44	46.25	**2.37**	2.45	2.63	2.56	2.62	3.30
ER random graph	48.27	12.48	65.13	**10.97**	23.56	36.75	2.70	**2.52**	2.93	2.92	2.78	2.55
SBM network	**34.07**	47.14	49.03	54.98	44.56	40.70	2.55	**2.30**	2.78	2.86	2.73	2.69
Sensor network	32.47	**18.71**	33.45	21.14	39.28	36.75	2.67	**2.61**	3.50	3.42	3.06	2.85
Real-world network (Taiwan)	1.18	**0.55**	2.95	3.08	4.59	4.61	0.97	**0.65**	1.38	0.88	0.79	0.80

TABLE IV
THE EFFICIENCY COMPARISON, IN TERMS OF TIME REQUIRED (MEASURED IN SECONDS) FOR CONTACT TRACING, BETWEEN ALGORITHM DEEPTRACE AND OTHER BASELINE METHODS ACROSS EPIDEMIC NETWORKS OF VARYING SCALES

Methods	N = 64	N = 128	N=256	N=512
DeepTrace	$\mathbf{0.58}$	$\mathbf{5.83{\times}10^1}$	$\mathbf{7.39{\times}10^2}$	$\mathbf{1.12{\times}10^3}$
Rumor center	$2.32{\times}10^1$	$4.36{\times}10^2$	$1.03{\times}10^3$	$2.52{\times}10^4$
SCT	$4.32{\times}10^1$	$6.55{\times}10^2$	$6.22{\times}10^3$	$1.43{\times}10^5$

error, when the underlying network is a tree with a symmetrical structure, such as complete N-ary graphs or regular trees, DeepTrace+DFS performs better. In summary, DeepTrace generally exhibits lower average distance errors than the baseline methods. BFS tracing often performs better than DFS for DeepTrace, indicating that BFS may be more effective for these structures. The results show that DeepTrace achieves higher accuracy in complex network topologies and consistently outperforms baseline methods across various networks. We evaluated the computational efficiency of DeepTrace, Rumor Center, and STC, measuring the average contact tracing time using DFS and BFS. As shown in Table IV, DeepTrace achieves significantly higher computational speed than the other methods.

C. DeepTrace for COVID-19 Epidemic Networks

In this section, we conduct experiments on COVID-19 pandemic data in Taiwan and Hong Kong to evaluate the performance of *Algorithm DeepTrace*. A summary of these epidemic network datasets is presented in Table V. We refer to G_n as the epidemic network in the following, where nodes in G_n represent confirmed cases, and each edge in G_n implies that there is close contact between two confirmed cases. We compare *Algorithm DeepTrace* to estimators using (2), and since these two clusters are relatively small, the computation of the ML estimator (2) is feasible.

1) Contact Tracing for COVID-19 Pandemic in Taiwan: The COVID-19 Omicron is a variant of the SARS-CoV-2 virus first reported on 24 November 2021, has shorter incubation times and is known to be more contagious than previous variants (with an effective reproduction number of 18.6 as compared to the Alpha variant of 3) that has since become the predominant variant in circulation worldwide. The infectiousness of Omicron has led to huge infection graphs and severely undermined the capacity of contact tracing in many countries. We first conduct experiments using data on the early spread of Omicron of COVID-19in Taiwan from March 13, 2022, to April 1, 2022. The data is collected from the confirmed case reports released daily by the Taiwan Centers for Disease Control at https://www.cdc.gov.tw. Each node in the networks represents a confirmed case and is marked with the reported number of the confirmed case. Each edge represents a close contact between two confirmed cases.

Fig. 9 is the largest contact tracing network constructed with the collected data with 92 confirmed cases. Case 22595 is the reported superspreader in this contact tracing network. As Case 22595 is also the earliest discovered case, we consider it the index case for contact tracing. Using the trained GNN in Algorithm DeepTrace, we applied it to this network and detected that Case 22595, colored red, is the most likely superspreader. These findings align with the actual superspreaders reported by the Taiwanese authorities.

2) Contact Tracing for COVID-19 Pandemic in Hong Kong: Due to Hong Kong's strict "Zero-COVID" policy with rigorous quarantine and testing, its population remained largely unaffected by COVID-19 until January 2022. However, the arrival of the Omicron variant led to one of the world's highest death

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

TABLE V
A SUMMARY OF COVID-19 EPIDEMIC NETWORK DATASETS FOR ALGORITHM DEEPTRACE

Epidemic network	Nodes	Edges	Components	Ground truths	Avg. Error Hops to MLE	Avg. Err. Hops to ground truth
Taiwan (2022.03.13-2022.04.01)	92	117	1	Known	0	0
Hong Kong (2021.12.31-2022.01.22)	102	97	4	Known	0	1/4
Hong Kong (2022.01.31-2022.02.03)	402	390	11	Unknown	0	ground truth unknown

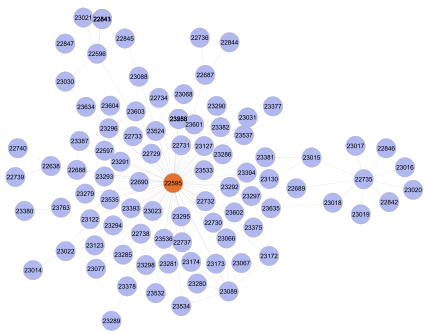


Fig. 9. Illustration of contact tracing with DeepTrace for a COVID-19 Omicron variant outbreak in Taiwan in early 2022. The blue nodes represent infected nodes, while the red nodes are the superspreaders predicted by DeepTrace, which incidentally correspond exactly to the actual superspreaders (i.e., ground truth) reported by the Taiwanese authorities.

tolls in early 2022, despite widespread vaccine access and Omicron's lower lethality [48], [49]. This unique scenario allows for real-world application of the SI spreading model and evaluation of Algorithm DeepTrace using data from Hong Kong's Omicron outbreak. We first consider an epidemic network with four connected components in Hong Kong from 31 December 2021 to 22 January, the data for which is from [49]. This epidemic network contains 102 SARS-CoV-2 Omicron infections associated with four imported cases (nodes 1, 29, 70, and 100) in the connected components, which serve as ground truths of superspreaders.

DeepTrace is used to iteratively identify the most likely superspreader, as outlined in (2), while the forward contact tracing network expands. Fig. 10 illustrates the epidemic network and contact tracing process. Initially, nodes 12, 36, 85, and 102 serve as index cases for the tracing process. As the network grows, DeepTrace identifies the most likely superspreaders, shifting from node 12 to 1, from 36 to 35, from 85 to 84 and then 70, and from 102 to 100 across the four connected components. The final detected superspreaders (nodes 1, 29, 70, and 100) correspond to the predictions from (2). Nodes 1, 29, and 100 are confirmed superspreaders, while node 70 is one hop away from the true superspreader, node 68. These results corroborate the efficiency of DeepTrace.

Next, we consider utilizing DeepTrace on another epidemic network in Hong Kong, where the ground truths of superspreaders are unknown. Following [48], we retrieved data of confirmed Omicron cases in Hong Kong from January 31, 2022, to February 3, 2022, from the Hong Kong government's public sector open data portal at https://data.gov.hk/en-data/dataset/hk-dh-

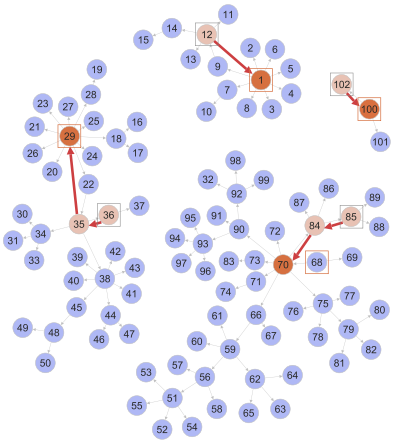


Fig. 10. Illustration of contact tracing with DeepTrace for an early Omicron outbreak in Hong Kong from 31 December 2021 to 22 January. The blue nodes represent infected nodes. The nodes in the gray boxes are considered the index cases of the Omicron outbreak. The nodes in the red boxes are the true superspreaders. The nodes in pink are superspreaders detected in the subgraph of the epidemic network during contact tracing. The nodes in red are the most likely superspreaders in the spreading clusters in the epidemic network. The red arrows indicate the movement of superspreaders detected by DeepTrace as contact tracing continues.

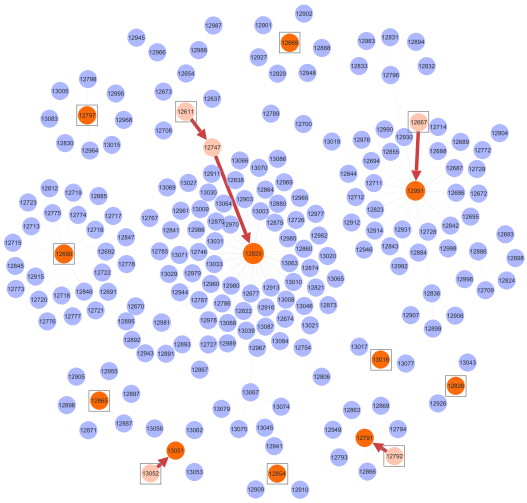


Fig. 11. Illustration of contact tracing using *Algorithm DeepTrace* for an early Omicron outbreak in Hong Kong in 2022. The blue nodes represent infected nodes. The nodes in the boxes are the index cases of the Omicron outbreak. The nodes in pink are superspreaders detected in the subgraph of the epidemic network during contact tracing. The nodes in red are the most likely superspreaders in the spreading clusters in the epidemic network. The red arrows indicate the movement of superspreaders detected by DeepTrace as contact tracing continues.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

chpsebcddr-novel-infectious-agent. We constructed epidemic networks using this data, where each node represents a confirmed case and edges connect nodes that visited the same building. The earliest reported case in [48] served as the initial index case for forward contact tracing, with DeepTrace iteratively identifying superspreaders as the network expanded.

Fig. 11 illustrates the contact tracing networks built from the retrieved data, showing how DeepTrace's iterative outputs converge on the most likely superspreader from (2) as the network expands. For instance, in the largest spreading cluster, case 12611 is the index case and initial superspreader. As the tracing network develops, DeepTrace identifies the superspreader, shifting from case 12611 to 12747 and finally to case 12825, which matches the prediction from (2). This demonstrates that DeepTrace effectively identifies likely superspreaders without prior knowledge of the true ones. Targeting these identified superspreaders with treatment or quarantine measures could help suppress further epidemic spread.

VI. Conclusion

Digital contact tracing remains a crucial challenge in the pandemic response. This paper formulates both forward and backward contact tracing strategies as maximum likelihood estimations of the epidemic source, which are essential for identifying superspreaders. Starting from an index case, contact tracers explore the epidemic network using breadth-first or depth-first search to compute the graph center of the contact tracing subgraph, leading to optimal likelihood estimates for specific topologies. We introduce Algorithm DeepTrace, which employs a graph neural network with semi-supervised learning, incorporating pre-training and fine-tuning phases for large graphs through message passing. This approach enhances large-scale graph inference in computational epidemiology and connects to online graph exploration, providing low-complexity greedy algorithms for contact tracing. We validate our methodology by modeling real COVID-19 superspreading events in Hong Kong and Taiwan, showing that our algorithm outperforms existing heuristics.

To enhance digital contact tracing and address challenges like Antimicrobial Resistance (AMR), future work will focus on advanced graph exploration and adaptive learning to handle new epidemic and AMR networks. We aim to develop efficient decomposition techniques for optimization, extend Algorithm DeepTrace to process real-time data streams, and improve scalability for larger networks. By integrating pathogen resistance data, our approach can optimize contact tracing and contribute to scalable, data-driven surveillance systems for pandemics and AMR mitigation.

Appendix

A. Proof of Theorem 1

Let $\mathbb{G}_N$ be a d-regular tree and $\overline{v} \in \mathbb{G}_N$ satisfies (5), we assume that the maximum distance from $\overline{v}$ to all leaves nodes is $k+1$. Then by (5), there are only two possible distances from $\overline{v}$ to all leaves, which are k and $k+1$. For simplicity, let $T_{d,k}$ denote the perfect d-regular tree with k levels, and we assume that the root level is 0. We denote t_v^u as the size of the subtree rooted at v after removing the edge $\{(u,v)\}$ from $\mathbb{G}_N$. We have the following properties of $\mathbb{G}_N$:

1) The topology of $\mathbb{G}_N$ is composed of $T_{d,k}$ plus some node v that satisfies $\text{dist}(\overline{v}, v) = k+1$ and v is connected to a leaf node in $T_{d,k}$, moreover, we have $\overline{v} = v_N^*$.
2) If $\overline{v}$ is the root of $\mathbb{G}_N$, then all subtree of $\mathbb{G}_N$ satisfy (5).

The proof of the above two properties can be found in the following subsections. Since $\mathbb{G}$ is a d-regular tree, finding the superspreader by (2) is equivalent to finding its centroids [17], i.e., v_n^* is the centroid of $\mathbb{G}_N$. Let $\overline{v}$ be the root of $\mathbb{G}_N$, then we can prove Theorem 1 by showing that if $v_i^* \neq v_{i+1}^*$, then v_{i+1}^* is the parent of v_i^*. To contrary, assume $\exists i < N$ such that $v_i^* \neq v_{i+1}^*$ and $v_{i+1}^* = v_c$, where v_c is a child of v_i^*. Without loss of generality, we can assume that v_i^* is at the lth level of $\mathbb{G}_N$. Then, from the above property 1, we can deduce that $t_{v_c}^{v_i^*} > t_{v_i^*}^{v_c}$ since v_c is the centroid of $\mathbb{G}_N$. Note that the maximum size of $t_{v_c}^{v_i^*}$ is $\frac{(d-1)^{k-l+1}-1}{d-2}$ since the subtree rooted at v_c also satisfies (5) by the above property 2. Similarly, since the BFS algorithm discovers all vertices at a distance $k-l$ from v_i^*, we have the minimum size of $t_{v_i^*}^{v_c} = (d-2)\frac{(d-1)^{k-l}-1}{d-2} + \frac{(d-1)^{k-l}-1}{d-2} + 1$, where the first term of this equation is the sum of the size of all other subtrees rooted at v_i^* except the one rooted at v_c. The second term is the size of $t_{v_p}^{v_i^*}$ where v_p is the parent of v_i^*, and the third term is v_i^* itself. Hence, we have $t_{v_c}^{v_i^*} = t_{v_i^*}^{v_c}$ which contradicts the assumption that the v_c is the centroid of $\mathbb{G}_N$. We can conclude that v_{i+1}^* must be the parent of v_i^* if $v_i^* \neq v_{i+1}^*$ which implies that the trajectory of v_n^* is the shortest path to the root $v_N^* = \overline{v}$.

B. Proof of Property 1 of $\mathbb{G}_N$ Used in Theorem 1

Let $\mathbb{G}_N$ be a d-regular tree and $\overline{v} \in \mathbb{G}_N$ satisfies (5), we assume that the maximum distance from $\overline{v}$ to all leaves nodes is $k+1$. Then by (5), there are only two possible distances from $\overline{v}$ to all leaves, which are k and $k+1$. By the definition of d-regular tree and (5), we can deduce that for all $v \in \mathbb{G}_N$ $\text{dist}(v, \overline{v}) < k$, v is not a leaf of $\mathbb{G}_N$ which implies the degree of v must be d. Hence, $\mathbb{G}_N$ contains a d-regular tree with k levels denoted as $T_{d,k}$. The rest of the nodes in $\mathbb{G}_N \backslash T_{d,k}$ are leaves in $\mathbb{G}_N$ with distance k or $k+1$ from the node $\overline{v}$. Since the topology of $\mathbb{G}_N$ is composed of $T_{d,k}$ and an additional leaf-level, we can conclude that $\overline{v} = v_N^*$ [10], [17].

C. Proof of Property 2 of $\mathbb{G}_N$ Used in Theorem 1

Let $\mathbb{G}_N$ be a d-regular tree and $\overline{v} \in \mathbb{G}_N$ satisfies (5), we assume that the maximum distance from $\overline{v}$ to all leaves nodes is $k+1$. Let $\overline{v}$ be the root of $\mathbb{G}_N$, and $v \in \mathbb{G}_N$ be an arbitrary node. If v is a leaf or the root of $\mathbb{G}_N$, then the subtree rooted at v satisfies (5). If v is not a leaf or the root of $\mathbb{G}_N$, then we prove this property by contradiction. Assume that the subtree rooted at v, denoted as $T_v^{\overline{v}}$ does not satisfy (5), then there are two leaves $u, w \in T_v^{\overline{v}}$, satisfy $|\text{dist}(u,v) - \text{dist}(w,v)| > 1$. since $\mathbb{G}_N$ is a tree, we have $\text{dist}(\overline{v}, u) = \text{dist}(\overline{v}, v) + \text{dist}(v, u)$ and

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

$\text{dist}(\overline{v}, w) = \text{dist}(\overline{v}, v) + \text{dist}(v, w)$ which implies $|\text{dist}(\overline{v}, u) - \text{dist}(\overline{v}, w)| > 1$ and contradicts to our assumption.

D. Proof of Theorem 2

We can prove this theorem by showing that if $(v^*_{j_1} = v, v^*_{j_1+1} = u) \in S_2$ and $(v^*_{j_2} = v, v^*_{j_2+1} = w) \in S_2$ where $j_2 > j_1$, then we have $j_2 > 2j_1$. Assume there are two pairs $(u, v), (w, v) \in S_3$ where $u, v, w \in \mathbb{G}_N$. Since each pair in S_3 must have a corresponding pair in S_2, there are $(v^*_{j_1} = v, v^*_{j_1+1} = u) \in S_2$ and $(v^*_{j_2} = v, v^*_{j_2+1} = w) \in S_2$. If we treat the index case v^1 as the root of $\mathbb{G}_N$, then we have $v^*_{j_1}$ is the parent of $v^*_{j_1+1}$ and $v^*_{j_2+1}$. Let u be the d_1th node and w be the d_2th node visited by the DFS strategy respectively, i.e., $v^*_{j_1+1} = v^{d_1}$ and $v^*_{j_2+1} = v^{d_2}$ where $d_2 > d_1$.

Note that if a node in $\mathbb{G}_N$ with weight strictly less than $n/2$, then the node is the unique ML estimator on contact tracing network $\mathbb{G}_N$ [15], [16], [17]. Hence we have $j_1 + 1 = 2d_1 - 1$.. Moreover, since $(v = v^*_{j_2}, v^*_{j_2+1}) \in S_2$, we have

$$j_2 + 1 = 2d_2 - 1 > 2(j_1 + 1) - 1 = 2j_1 + 1.$$

The second inequality follows by $d_2 > j_1 + 1$ since w will not be visited until the subtree $T^v_u \subseteq \mathbb{G}_N$ is fully visited by the DFS algorithm. We can conclude that if v is involved in more that one pairs say (u, v) and $(w, v) \in S_3$ then there are corresponding pairs $(v = v^*_{j_1}, u = v^*_{j_1+1})$, and $(v = v^*_{j_2}, w = v^*_{j_2+1}) \in S_2$ in S_2 with index $j_2 > 2j_1$. From Lemma 1, we have $|S_2| \leqslant (N-1)/2$, which implies v cannot be involved in more than $\log_2 \frac{N-1}{2}$ pairs in S_3, since each pair $(u, v) \in S_3$ must have a corresponding $(v, u) \in S_2$.

E. Proof of Lemma 1

Since the DFS contact tracing starts from the index case v^1, we can assume that the index case v^1 is the root of the DFS tree. Now, we can distinguish parent-node and child-node in each pair $(v^*_j, v^*_{j+1}) \in S$. Due to the property of the DFS tracing, a node is fully visited when all its descendants are fully visited; we have the following observation: If $(v^*_j = u, v^*_{j+1} = v) \in S_2 \cup S_3$ and v is the parent of u, then there is no other $i > j$ such that $(v^*_i = v, v^*_{i+1} = u) \in S_2 \cup S_3$. Moreover, we can observe that:

1) If $(u, v) \in S_2$, then u is the parent node of v.
2) If $(u, v) \in S_3$, then u is the child of v.

Since the estimated superspreader changes from u to v only when $P(\mathbb{G}_N \mid v) > P(\mathbb{G}_N \mid u)$. We can conclude that both type two and type three transitions must be followed by a type one transition, which means the size of the type one transition is more than type two and type three. Hence, we have $|S_1| \geqslant |S_2| + |S_3|$. Moreover, from the above two properties, we have the fact that for each $(u, v) \in S_3$, there is a corresponding $(v, u) \in S_2$, i.e., $|S_2| \geqslant |S_3|$. Combining the results above, we have $|S_1| \geqslant |S_2| \geqslant |S_3|$. Since $|S_1| + |S_2| + |S_3| = N - 1$ and $|S_2| \geqslant |S_3|$, we have $|S_1| \geqslant \frac{N-1}{2}$ and $|S_3| \leqslant \frac{N-1}{4}$.

F. Proof of Theorem 3

As shown in (2), solving the MLE problem needs the information of all the permitted permutations for all the nodes in G_n. Since a newly infected node can only be infected by one of its infected neighbors, v_{i+1} must be the neighbor of v_i in any permitted permutation σ, and thus calculating $p(\sigma \mid v)$ with (4) is a recursive process of aggregating the information of degree $d(v_l)$ and Φ_l from the lth layer's neighbors of node v in σ, which can be expressed as

$$\mathbb{P}^{(l)}(\sigma \mid v) = \sum_{u \in N_G(v)} f\left(\mathbb{P}^{(l-1)}(\sigma \mid v), \mathbb{P}^{(l-1)}(\sigma \mid u)\right), \quad (12)$$

with $\mathbb{P}^{(l)}(\sigma \mid v)|_{l=1} = 1/d(v)$. Therefore, it is a recursive iteration to calculate $\sum_{\sigma \in \Omega(G_n|v)} \mathbb{P}(\sigma|v)$ to aggregate the information of degree $d(v_l)$ and Φ_l from its lth layer's neighbors in all $\sigma \in \Omega(G_n \mid v)$, which can be shown as

$$\begin{aligned} &\mathbb{P}^{(l)}(G_n \mid v) \\ &= \sum_{\sigma \in \Omega(G_n|v)} \sum_{u \in N_G(v)} f\left(\mathbb{P}^{(l-1)}(\sigma|v), \mathbb{P}^{(l-1)}(\sigma|u)\right) \\ &= \sum_{u \in N_G(v)} g\left(\mathbb{P}^{(l-1)}(G_n|v), \mathbb{P}^{(l-1)}(G_n|u)\right). \end{aligned} \quad (13)$$

Note that (13) demonstrates the process of iterative aggregation to calculate $\mathbb{P}(G_n \mid v)$ for node v. Since $\mathbb{G}$ is connected, the shortest distance between any two nodes satisfies $\text{dist}(u, v) \leqslant \text{diam}(G)$. Thus, after some $\text{diam}(G)$ steps of aggregation and combination, each node can obtain the feature information of all nodes in the graph. Furthermore, it is evident that both $f(\cdot)$ and $g(\cdot)$ are continuous functions. Consequently, based on the universal approximation theorem, these functions can be approximated to any desired degree of precision using standard multilayer perceptrons with hidden layers and non-constant, monotonically increasing activation functions [44].

Acknowledgment

C. W. Tan acknowledges helpful discussions on the subject with O. T. Ng and K. Marimuthu of the Singapore National Centre for Infectious Diseases.

References

[1] S. Landau, *People Count: Contact-Tracing Apps and Public Health*. Cambridge, MA, USA: MIT Press, 2021.
[2] Y. Bengio et al., "Predicting infectiousness for proactive contact tracing," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–21.
[3] D. Leith and S. Farrell, "Measurement-based evaluation of Google/Apple Exposure Notification API for proximity detection in a commuter bus.," *PLoS ONE*, vol. 16, no. 4, pp. 1–16, 2021.
[4] H. Stevens and M. B. Haines, "Tracetogether: Pandemic response, democracy, and technology," *East Asian Sci., Technol. Soc.*, vol. 14, no. 3, pp. 523–532, 2020.
[5] Z. Fei, Y. Ryeznik, A. Sverdlov, C. W. Tan, and W. K. Wong, "An overview of healthcare data analytics with applications to the COVID-19 pandemic," *IEEE Trans. Big Data*, vol. 8, no. 6, pp. 1463–1480, Dec. 2022.
[6] D. C. Adam et al., "Clustering and superspreading potential of SARS-CoV-2 infections in Hong Kong," *Nature Med.*, vol. 26, no. 11, pp. 1714–1719, 2020.
[7] G. Cencetti et al., "Digital proximity tracing on empirical contact networks for pandemic control," *Nature Commun.*, vol. 12, no. 1, 2021, Art. no. 1655.
[8] S. Kojaku, L. Hébert-Dufresne, E. Mones, S. Lehmann, and Y.-Y. Ahn, "The effectiveness of backward contact tracing in networks," *Nature Phys.*, vol. 17, no. 5, pp. 652–658, 2021.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

[9] W. J. Bradshaw, E. C. Alley, J. H. Huggins, A. L. Lloyd, and K. M. Esvelt, "Bidirectional contact tracing could dramatically improve COVID-19 control," *Nature Commun.*, vol. 12, no. 1, 2021, Art. no. 232.

[10] D. Shah and T. Zaman, "Rumors in a network: Who's the culprit?," *IEEE Trans. Inf. Theory*, vol. 57, no. 8, pp. 5163–5181, Aug. 2011.

[11] N. Bailey, *The Mathematical Theory of Infectious Diseases and Its Applications*. New York, NY, USA: Hafner Press, 1975.

[12] N. C. Grassly and C. Fraser, "Mathematical models of infectious disease transmission," *Nature Rev. Microbiol.*, vol. 6, no. 6, pp. 477–487, 2008.

[13] Y.-C. Chen, P.-E. Lu, C.-S. Chang, and T.-H. Liu, "A time-dependent SIR model for COVID-19 with undetectable infected persons," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 3279–3294, Oct.–Dec. 2020.

[14] R. Eletreby, Y. Zhuang, K. M. Carley, O. Yağan, and H. V. Poor, "The effects of evolutionary adaptations on spreading processes in complex networks," *Proc. Nat. Acad. Sci. United States Amer.*, vol. 117, no. 11, 2020, Art. no. 5664.

[15] D. Shah and T. Zaman, "Rumor centrality: A universal source detector," in *Proc. ACM SIGMETRICS/PERFORMANCE Joint Int. Conf. Meas. Model. Comput. Syst.*, 2012, pp. 199–210.

[16] B. Zelinka, "Medians and peripherians of trees," *Archivum Mathematicum*, vol. 4, no. 2, pp. 87–95, 1968.

[17] P. D. Yu, C. W. Tan, and H. L. Fu, "Averting cascading failures in networked infrastructures: Poset-constrained graph algorithms," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 4, pp. 733–748, Aug. 2018.

[18] C. W. Tan and P.-D. Yu, "Contagion source detection in epidemic and infodemic outbreaks: Mathematical analysis and network algorithms," *Foundations Trends Netw.*, vol. 13, no. 2–3, pp. 107–251, 2023.

[19] L. Vassio, F. Fagnani, P. Frasca, and A. Ozdaglar, "Message passing optimization of harmonic influence centrality," *IEEE Trans. Control Netw. Syst.*, vol. 1, no. 1, pp. 109–120, Mar. 2014.

[20] N. Karamchandani and M. Franceschetti, "Rumor source detection under probabilistic sampling," in *Proc. IEEE Int. Symp. Inf. Theory*, 2013, pp. 2184–2188.

[21] Z. Wang, W. Dong, W. Zhang, and C. W. Tan, "Rumor source detection with multiple observations: Fundamental limits and algorithms," in *Proc. ACM SIGMETRICS Perform. Eval. Rev.*, 2014, pp. 1–13.

[22] W. Luo, W. P. Tay, and M. Leng, "Identifying infection sources and regions in large networks," *IEEE Trans. Signal Process.*, vol. 61, no. 11, pp. 2850–2865, Jun. 2013.

[23] P.-D. Yu, C. W. Tan, and H.-L. Fu, "Epidemic source detection in contact tracing networks: Epidemic centrality in graphs and message-passing algorithms," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 2, pp. 234–249, Feb. 2022.

[24] A.. Sridhar and H. V. Poor, "Sequential estimation of network cascades," in *Proc. Asilomar Conf. Signals, Syst., Comput.*, 2020, pp. 1507–1511.

[25] E. A. Meirom, C. Milling, C. Caramanis, S. Mannor, S. Shakkottai, and A. Orda, "Localized epidemic detection in networks with overwhelming noise," in *Proc. 2015 ACM SIGMETRICS Int. Conf. Meas. Model. Comput. Syst.*, 2015, pp. 441–442.

[26] A. Sridhar and H. V. Poor, "Bayes-optimal methods for finding the source of a cascade," in *Proc. IEEE Int. Conf. Acoustics, Speech, Signal Process.*, 2021, pp. 5190–5194.

[27] P.-D. Yu, L. Zheng, and C. W. Tan, "Contagion source detection by maximum likelihood estimation and starlike graph approximation," in *Proc. 58th Annu. Conf. Inf. Sci. Syst.*, 2024, pp. 1–6.

[28] V. La Gatta, V. Moscato, M. Postiglione, and G. Sperli, "An epidemiological neural network exploiting dynamic graph structured data applied to the COVID-19 outbreak," *IEEE Trans. Big Data*, vol. 7, no. 1, pp. 45–55, Mar. 2021.

[29] C. Shah et al., "Finding patient zero: Learning contagion source with graph neural networks," 2020, *arXiv:2006.11913*.

[30] X. Ru, J. M. Moore, X.-Y. Zhang, Y. Zeng, and G. Yan, "Inferring patient zero on temporal networks via graph neural networks," *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 8, pp. 9632–9640.

[31] S. Chen, P.-D. Yu, C. W. Tan, and H. V. Poor, "Identifying the superspreader in proactive backward contact tracing by deep learning," in *Proc. 56th Annu. Conf. Inf. Sci. Syst.*, 2022, pp. 43–48.

[32] K. K.-W. To et al., "Temporal profiles of viral load in posterior oropharyngeal saliva samples and serum antibody responses during infection by SARS-CoV-2: An observational cohort study," *Lancet Infect. Dis.*, vol. 20, no. 5, pp. 565–574, 2020.

[33] A. Ganesh, L. Massoulie, and D. Towsley, "The effect of network topology on the spread of epidemics," in *Proc. IEEE 24th Annu. Joint Conf. IEEE Comput. Commun. Societies.*, 2005, vol. 2, pp. 1455–1466.

[34] B. Awerbuch and R. Gallager, "A new distributed algorithm to find breadth first search trees," *IEEE Trans. Inf. Theory*, vol. 33, no. 3, pp. 315–322, May 1987.

[35] S. Even, *Graph Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2011.

[36] B. Kalyanasundaram and K. Pruhs, "Constructing competitive tours from local information," *Theor. Comput. Sci.*, vol. 130, pp. 125–138, 1994.

[37] N. Megow, K. Mehlhorn, and P. Schweitzer, "Online graph exploration: New results on old and new algorithms," *Theor. Comput. Sci.*, vol. 463, pp. 62–72, 2012.

[38] H. Dai, Y. Li, C. Wang, R. Singh, P.-S. Huang, and P. Kohli, "Learning transferable graph exploration," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 2518–2529.

[39] I. Chiotellis and D. Cremers, "Neural online graph exploration," 2020, *arXiv:2012.03345*.

[40] D. Acemoglu, A. Malekian, and A. Ozdaglar, "Network security and contagion," *J. Econ. Theory*, vol. 166, pp. 536–585, 2016.

[41] N. E. Friedkin, "Theoretical foundations for centrality measures," *Amer. J. Sociol.*, vol. 96, no. 6, pp. 1478–1504, 1991.

[42] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–14.

[43] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS' 17)*, 2017, pp. 1025–1035.

[44] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Netw.*, vol. 4, no. 2, pp. 251–257, 1991.

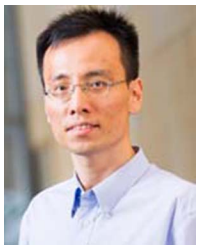
[45] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python Sci. Conf.*, 2008, pp. 11–15.

[46] P.-D. Yu, C. W. Tan, and H.-L. Fu, "Rumor source detection in finite graphs with boundary effects by message-passing algorithms," in *Proc. 2017 IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2017, pp. 86–90.

[47] M. Chen et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.

[48] R. Xie et al., "Resurgence of omicron BA.2 in SARS-CoV-2 infection-naive Hong Kong," *Nature Commun.*, vol. 14, 2023, Art. no. 2422.

[49] Y. M. Mefsin et al., "Epidemiology of infections with SARS-CoV-2 Omicron BA. 2 variant, Hong Kong, January–March 2022," *Emerg. Infect. Dis.*, vol. 28, no. 9, 2022, Art. no. 1856.

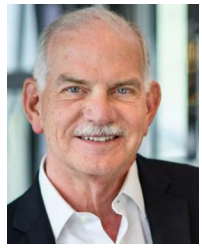


Chee Wei Tan (Senior Member, IEEE) received the M.A. and Ph.D. degrees in electrical engineering from Princeton University, Princeton, NJ, USA, in 2006 and 2008, respectively. He is currently an Associate Professor of computer science with Nanyang Technological University, Singapore. His research interests include networks, distributed optimization, and generative AI. He was the Editor of IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON COMMUNICATIONS, and as IEEE ComSoc Distinguished Lecturer. He was the recipient of the Princeton University Wu Prize for Excellence, Google Faculty Award, Teaching Excellence awards, and was selected twice for the U.S. National Academy of Engineering China-America Frontiers of Engineering Symposium.

Pei-Duo Yu received the B.Sc. and M.Sc. degrees in applied mathematics from the National Chiao Tung University, Taiwan, in 2011 and 2014, respectively, and the Ph.D. degree from the Department of Computer Science, City University of Hong Kong, Hong Kong. He is currently an Assistant Professor with the Department of Applied Mathematics, Chung Yuan Christian University, Taiwan. His research interests include combinatorics counting, graph algorithms, and optimization theory and its applications.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.

Siya Chen received the B.Sc. and M.Sc. degrees in applied mathematics from Shenzhen University, Shenzhen, China. She is currently working toward the Ph.D. degree with the Department of Computer Science, City University of Hong Kong, Hong Kong. Her research interests include machine learning and optimization theory in wireless network optimization.



H. Vincent Poor (Life Fellow, IEEE) received the Ph.D. degree in EECS from Princeton University, Princeton, NJ, USA, in 1977. From 1977 to 1990, he was with the Faculty of the University of Illinois at Urbana-Champaign, Champaign, IL, USA. Since 1990, he has been on the Faculty with Princeton, where he is currently the Michael Henry Strater University Professor. During 2006–2016, he was the Dean with the Princeton's School of Engineering and Applied Science, and he has also held visiting appointments at several other universities, including most recently at Berkeley and Caltech. His research interests include the areas of information theory, machine learning and network science, and their applications in wireless networks, energy systems and related fields. Among his publications in these areas is the book *Machine Learning and Wireless Communications* (Cambridge University Press, 2022). He is also a Member of the National Academy of Engineering and the National Academy of Sciences and is a Foreign Member of the Royal Society and other National and International academies. He was the recipient of the IEEE Alexander Graham Bell Medal in 2017.

Authorized licensed use limited to: SHIV NADAR UNIVERSITY. Downloaded on January 23,2026 at 06:47:17 UTC from IEEE Xplore. Restrictions apply.